Universidad Nacional
Escuela de Informática
Programación 4 (EIF209) — II Ciclo 2026

Documentación Técnica

Sistema Web para la Gestión de un Gimnasio

Gimnasio PowerFit

Grupo 1

Christopher Blanco Solano
Alexander Dittel Escobar
Aslehy Claret Aguilar Perez

Profesora: Marianella Solano Orias
Octubre 2026

Contenido

1. Descripción general del sistema
2. Arquitectura
3. Diagrama de componentes
4. Diagrama de despliegue
5. Casos de uso
6. Seguridad
7. Instrucciones de instalación

1. Descripción general del sistema

Gimnasio PowerFit es un sistema web para la gestión de un gimnasio. Los visitantes consultan la información del gimnasio, envían consultas y se inscriben a una rutina o a un curso. Cada usuario inicia sesión según su rol:

- **Cliente:** ve únicamente sus propias rutinas, consulta y actualiza su inscripción y puede desinscribirse.
- **Instructor:** gestiona las máquinas y las rutinas de los clientes.
- **Administrador:** gestiona los usuarios del sistema (clientes, instructores y administradores), los cursos y los parámetros del sistema.

Todas las vistas son parte de una única aplicación (SPA) y toda la información se obtiene de la base de datos MySQL mediante servicios RESTful; además se ofrece un servicio SOAP de ejemplo para consultar cursos.

Tecnologías

Capa	Tecnología
Frontend (SPA)	React 19, React Router 7, Vite 8 (servidor de desarrollo Node.js, puerto 5173)
Backend	Java 26, Spring Boot 4.1 (Spring Web MVC, Tomcat embebido, puerto 8080), Bean Validation, Spring Web Services (SOAP)
Seguridad	Spring Security: sesión HTTP, BCrypt, autorización por roles con `@PreAuthorize`, CORS y cabeceras de seguridad
Persistencia (ORM)	Spring Data JPA con Hibernate
Base de datos	MySQL 8 o superior, base `gimnasio_db` (script `database/GIMNASIO.sql`)
Control de versiones	Git y GitHub, rama `main` protegida con Pull Requests

2. Arquitectura

El sistema sigue una arquitectura cliente-servidor en tres capas. El navegador ejecuta la SPA de React, que se comunica con el backend únicamente mediante servicios RESTful en JSON. El backend aplica el patrón Modelo–Vista–Controlador y separa la lógica en capas; la persistencia se hace exclusivamente por medio del ORM.

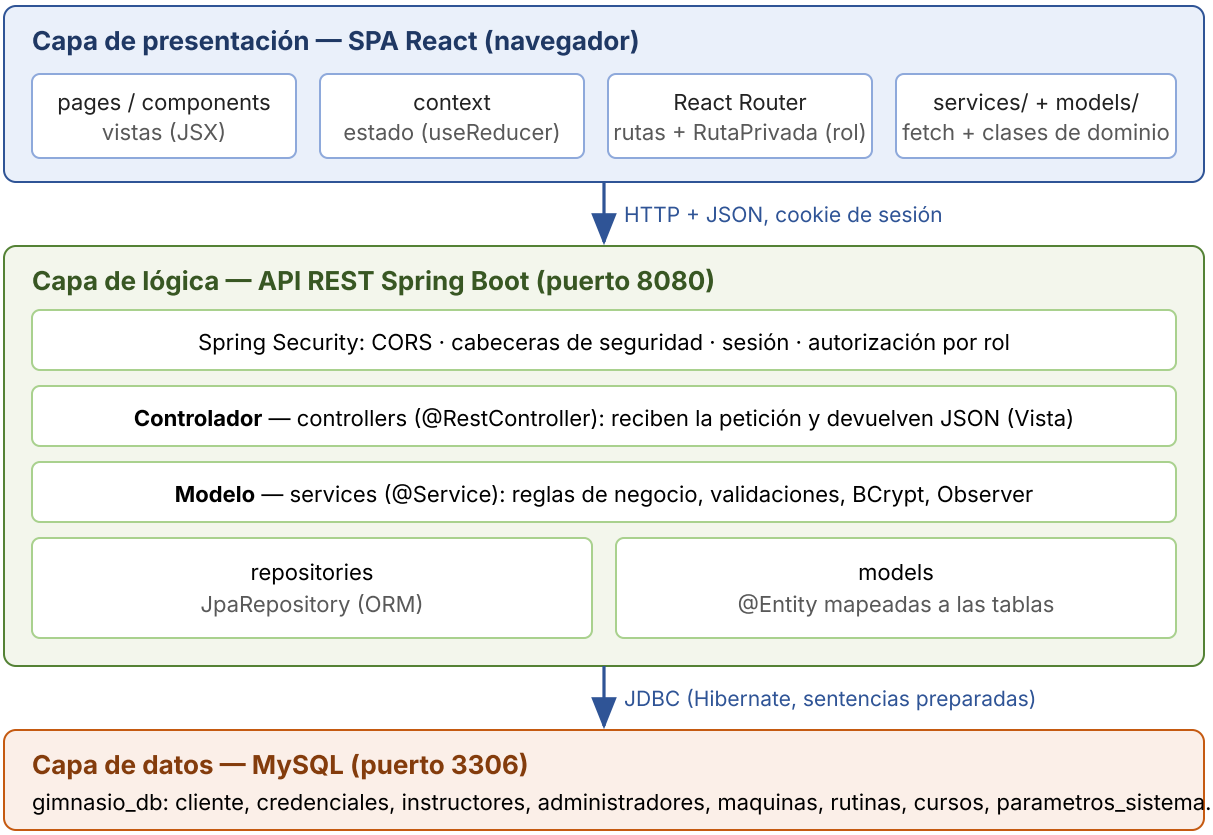


Figura 1. Arquitectura en capas del sistema

Responsabilidades por capa

Elemento	Responsabilidad
`controllers`	Exponen los servicios RESTful (GET, POST, PUT, DELETE), validan la entrada (`@Valid`) y devuelven JSON con el código HTTP correcto. Cada controlador declara qué rol puede usarlo con `@PreAuthorize`.
`services` / `services/impl`	Interfaces e implementaciones con la lógica de negocio: cupos, duplicados, edad mínima, cifrado de contraseñas, reglas de cada rol y las reglas de integridad del proyecto.
`repositories`	Interfaces `JpaRepository`: todas las lecturas y escrituras a la base de datos pasan por aquí.

`models`	Clases `@Entity` mapeadas a las tablas de `gimnasio_db`.
`dto`	Datos que entran y salen de la API (records) con sus validaciones (`@NotBlank`, `@Size`, `@Pattern`, `@Email`…). Nunca exponen hashes de contraseñas.
`exceptions`	Excepciones de negocio y `ManejadorGlobalExcepciones` (`@RestControllerAdvice`), que convierte todo error en JSON.
`config`	`SecurityConfig` (seguridad y cabeceras), `CorsConfig` (CORS), `SoapConfig` (WSDL) y `CargaInicialDatosRunner` (contraseñas de prueba).
`observer`, `sesion`, `soap`	Patrón Observer de inscripciones, registro Singleton de sesiones y el servicio SOAP de cursos.
Frontend `services/` y `models/`	Una función por servicio REST sobre `client.js` (fetch con la cookie de sesión y mensajes de error entendibles); los modelos (`Curso`, `Rutina`, `Maquina`, `Usuario`…) convierten el JSON en objetos.
Frontend `context/` y `hooks/`	Estado global con `useReducer` (sesión, cursos, notificaciones) y el hook `useRecursos` para las pantallas de mantenimiento.

Patrones de diseño

Patrón	Dónde se aplica
MVC	`models` (modelo), `controllers` (controlador) y las respuestas JSON consumidas por las vistas de React.
Singleton	`sesion/GestorSesiones.getInstancia()` y los servicios con `@Scope(SINGLETON)`.
Observer	`PublicadorInscripciones` notifica a `BitacoraInscripciones` y `EstadisticasInscripciones` en cada inscripción o desinscripción.
Iterator	Los `listar()` de los servicios recorren las entidades con `java.util.Iterator`; en el frontend, `models/Coleccion.js` implementa `[Symbol.iterator]` y se usa en tablas y filtros.

3. Diagrama de componentes

Muestra los componentes principales y las interfaces por las que se comunican. El frontend solo conoce la API REST; el backend solo accede a la base de datos mediante los repositorios JPA.

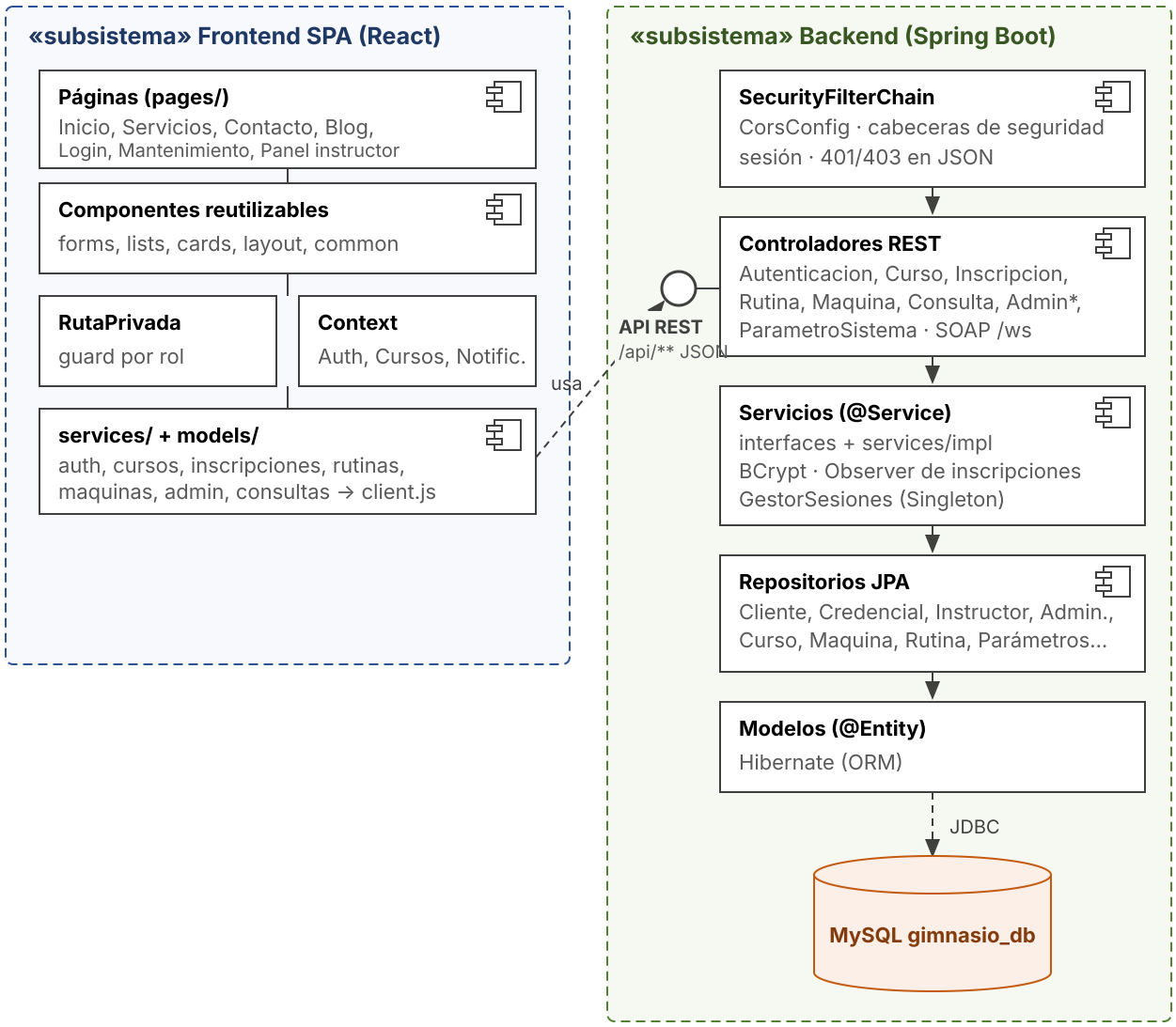


Figura 2. Diagrama de componentes

4. Diagrama de despliegue

En el entorno del proyecto los tres servidores corren en la misma computadora, cada uno en su propio puerto. El navegador descarga la SPA desde Vite y luego llama directamente a la API de Spring Boot; por eso el backend habilita CORS para el origen `http://localhost:5173`.

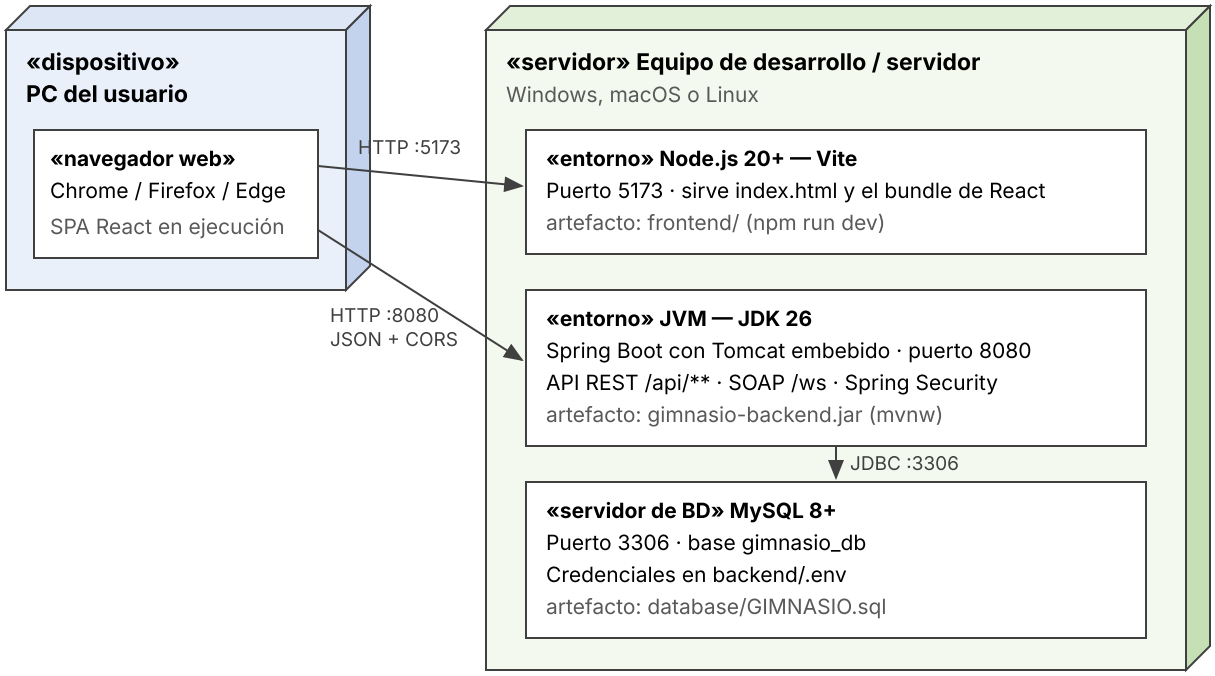


Figura 3. Diagrama de despliegue

Nodo / entorno	Puerto	Qué ejecuta
Navegador	—	La SPA de React; guarda la cookie de sesión JSESSIONID (http-only).
Node.js + Vite	5173	Sirve el frontend; agrega X-Frame-Options, X-Content-Type-Options y Referrer-Policy.
JVM + Spring Boot	8080	La API REST, la seguridad (sesión, roles, CORS, cabeceras) y el ORM.
MySQL	3306	La base de datos `gimnasio_db`.

> En producción el mismo esquema se mantiene, pero publicando el sitio y la API con HTTPS. En ese caso Spring Security envía además la cabecera `Strict-Transport-Security` (HSTS), ya configurada.

5. Casos de uso

Actores: **Visitante** (sin sesión), **Cliente**, **Instructor** y **Administrador**. Los tres últimos especializan al actor Usuario autenticado, que puede iniciar y cerrar sesión.

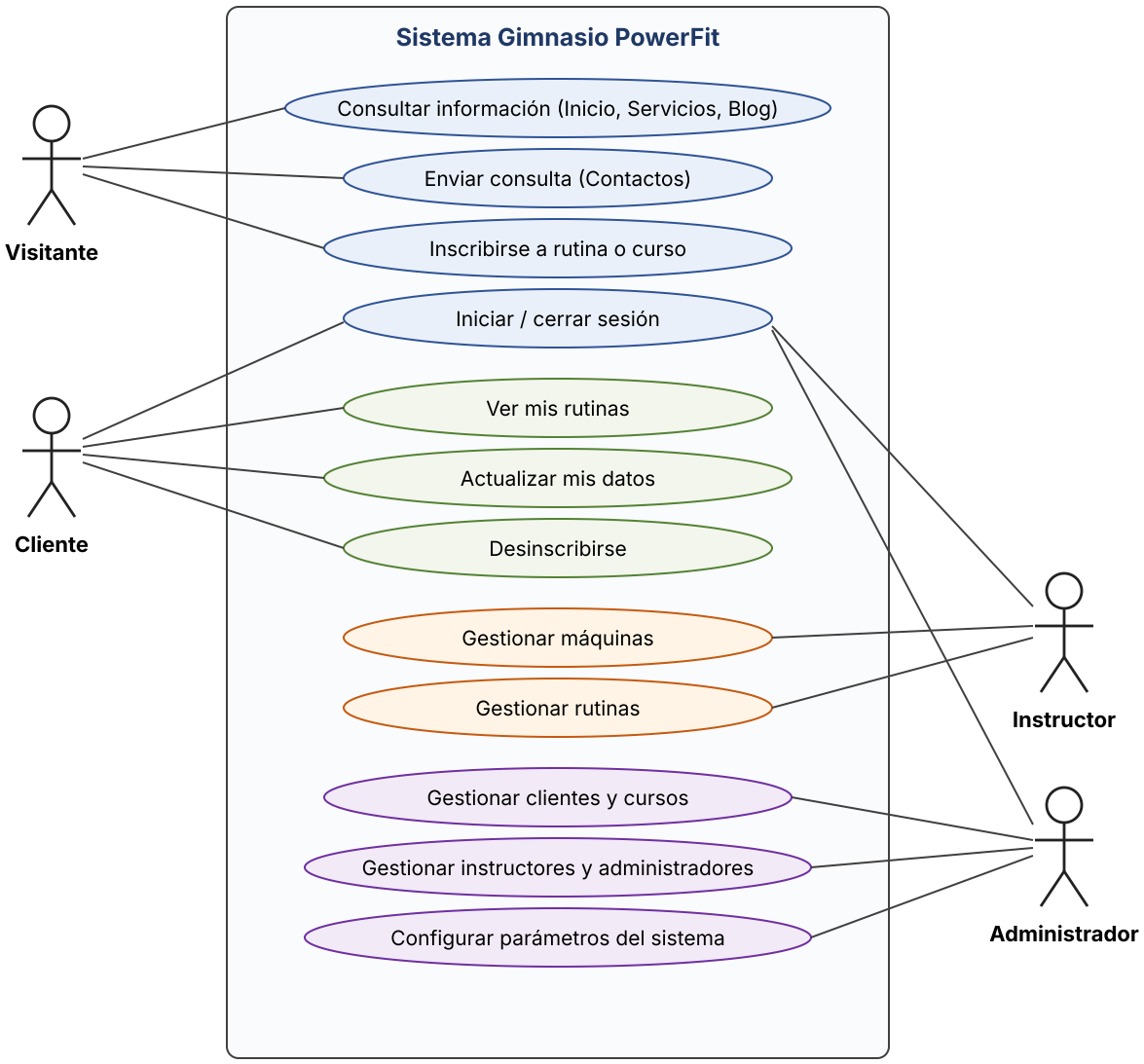


Figura 4. Diagrama de casos de uso

Descripción de los casos de uso

Caso de uso	Actor	Descripción y servicio REST
Consultar información	Todos	Ver Inicio, Servicios (cursos, máquinas, instructores) y Blog. `GET /api/cursos`, `GET /api/maquinas` (públicos).
Enviar consulta	Visitante	Completa el formulario de Contactos, que se valida en el cliente y en el servidor. `POST /api/consultas`.
Inscribirse	Visitante	Se registra con cédula y contraseña (cifrada con BCrypt) a una rutina o un curso. `POST /api/inscripciones`; responde 409 si la cédula ya existe.
Iniciar / cerrar sesión	Cliente, Instructor, Administrador	Con cédula o código y contraseña; el servidor crea la sesión HTTP y la cookie JSESSIONID. `POST /api/auth/login`, `GET /api/auth/sesion`, `POST /api/auth/logout`.
Ver mis rutinas	Cliente	Solo sus propias rutinas. `GET /api/rutinas/mias`, `GET /api/inscripciones/mia`.
Actualizar mis datos / desinscribirse	Cliente	Solo sobre su propia cédula (otra cédula responde 403); la pantalla Desinscribirse toma la cédula de la sesión. `PUT` y `DELETE /api/inscripciones/{cedula}`.
Gestionar máquinas	Instructor	Crear, editar y eliminar máquinas (no se elimina una máquina usada en rutinas: 409). `POST`, `PUT`, `DELETE /api/maquinas`.
Gestionar rutinas	Instructor	Asignar rutinas a los clientes (cliente, máquina, fecha y horas); el instructor es el de la sesión. Al editar solo cambian fecha y horas. `/api/rutinas`.
Gestionar clientes y cursos	Administrador	Listar, editar y eliminar clientes inscritos; crear, editar y eliminar cursos. `/api/inscripciones`, `/api/cursos`.
Gestionar instructores y administradores	Administrador	Con contraseña inicial cifrada; no se puede eliminar la propia cuenta ni el último administrador. `/api/admin/instructores` y `/api/admin/administradores`.
Configurar parámetros	Administrador	Consultar y modificar los parámetros del sistema. `GET` y `PUT /api/admin/parametros`.

Si un usuario sin sesión llama a un servicio protegido, la respuesta es **401**; si tiene sesión pero no el rol requerido, **403**. En el frontend, `RutaPrivada` redirige a `/login` sin sesión y muestra “Acceso denegado” cuando el rol no corresponde; después del login, cada rol llega a su sección.

6. Seguridad

6.1 Autenticación y autorización

- **Contraseñas:** se guardan cifradas con BCrypt (hash + salt) mediante el `PasswordEncoder` de Spring Security; nunca en texto plano. Clientes en `credenciales.password_hash`; instructores y administradores en la columna `password_hash` de su tabla.
- **Inicio de sesión:** un único endpoint (`POST /api/auth/login`) prueba el usuario como cliente (cédula), instructor (código) y administrador (código). El mensaje de error es el mismo si el usuario no existe o la contraseña es incorrecta.
- **Sesión:** HttpSession con la cookie JSESSIONID, marcada como http-only (no accesible desde JavaScript), que expira tras 30 minutos de inactividad. Al iniciar sesión se renueva el identificador (`changeSessionId`, contra fijación de sesión) y se guarda el `SecurityContext` con el rol (`ROLE_CLIENTE`, `ROLE_INSTRUCTOR` o `ROLE_ADMINISTRADOR`).
- **Roles:** `@PreAuthorize` en los controladores. Por ejemplo, `hasRole('INSTRUCTOR')` en máquinas y rutinas, `hasRole('ADMINISTRADOR')` en `/api/admin/**`, y `#cedula == authentication.name` para que un cliente solo modifique sus propios datos.

6.2 Protección contra inyección SQL

El sistema no construye sentencias SQL concatenando texto. Todo el acceso a datos pasa obligatoriamente por el ORM:

- Los servicios usan únicamente los repositorios `JpaRepository` (`findById`, `findAll`, `save`, `deleteById`, `existsById`) y consultas derivadas como `RutinaRepository.findByCliente_CedulaOrderByFechaDesc(Long cedula)`.
- Hibernate traduce cada operación a una **sentencia preparada** (PreparedStatement) con parámetros `?`. El valor que escribe el usuario viaja aparte, como dato, y nunca forma parte del texto SQL, así que un intento como `' OR '1'='1` se busca literalmente y no altera la consulta.
- En el código no hay `createNativeQuery`, `JdbcTemplate`, `Statement` ni `@Query`: todas las consultas son métodos del repositorio.
- Además, los datos se validan antes de llegar a la base (tipos numéricos para cédulas y códigos, longitudes y formatos).

Evidencia: con `spring.jpa.show-sql=true` la consola del backend muestra las sentencias con parámetros, por ejemplo:

```
select r1_0.id_rutina,r1_0.cliente,r1_0.fecha,r1_0.horas,r1_0.instructor,r1_0.maquina
from rutinas r1_0 where r1_0.cliente=? order by r1_0.fecha desc
```

Para ver además el valor enviado en cada parámetro se activa, en `application.properties`, `logging.level.org.hibernate.orm.jdbc.bind=TRACE`.

6.3 CORS

El frontend (`http://localhost:5173`) y la API (`http://localhost:8080`) son orígenes distintos, así que el navegador solo permite las llamadas si el backend lo autoriza. `config/CorsConfig` define una única configuración, que Spring Security aplica con `http.cors(...)`:

Regla	Valor
Orígenes permitidos	Lista blanca en `app.cors.origenes-permitidos` (application.properties): localhost:5173, 127.0.0.1:5173, localhost y 127.0.0.1. Nunca `*`.
Métodos	GET, POST, PUT, DELETE y OPTIONS (preflight)
Encabezados permitidos / expuestos	`Content-Type`, `Accept` / `Location`, `X-Registros-Eliminados`
Credenciales	`allowCredentials(true)`: deja pasar la cookie de sesión
Caché del preflight	3600 segundos

Como CORS se aplica dentro del filtro de seguridad, también las respuestas 401 y 403 llevan los encabezados CORS, y el frontend puede mostrar el mensaje correcto en lugar de un error genérico de CORS.

6.4 Cabeceras HTTP de seguridad

Cabecera	Valor	Protege contra
`X-Frame-Options`	`DENY`	Clickjacking: nadie puede cargar la API o el sitio dentro de un iframe.
`X-Content-Type-Options`	`nosniff`	Que el navegador interprete un archivo con otro tipo (por ejemplo, ejecutar un JSON como script).
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	Conexiones sin cifrar. Solo se envía sobre HTTPS (“cuando aplique”); en desarrollo, con HTTP, no aparece.
`Content-Security-Policy`	`default-src 'none'; frame-ancestors 'none'`	Carga de recursos no autorizados desde las respuestas de la API.
`Referrer-Policy`	`no-referrer` (API) / `strict-origin-when-cross-origin` (frontend)	Filtración de la URL de origen a otros sitios.

En el backend se configuran en `SecurityConfig` (`http.headers(...)`); en el frontend, en `frontend/vite.config.js` (`server.headers` y `preview.headers`). Para comprobarlas:

```
curl -I http://localhost:8080/api/health
curl -I http://localhost:5173/
```

6.5 Manejo de errores e integridad de datos

Ninguna respuesta incluye trazas del sistema (`server.error.include-stacktrace=never`). Los errores de negocio, validación y seguridad se devuelven como JSON `{"mensaje": "...", "errores": {campo: mensaje}}` con el código HTTP correspondiente (400, 401, 403, 404, 409 o 500); los 401/403 que genera el propio filtro de Spring Security usan el mismo formato. Reglas de integridad aplicadas en los servicios:

- No se modifican llaves primarias ni foráneas (por ejemplo, al editar una rutina solo cambian la fecha y las horas): 400.
- No se elimina un registro con relaciones (curso con inscritos, máquina usada en rutinas, instructor con rutinas): 409 con la explicación.
- Cada actualización y eliminación confirma los registros afectados (`registrosAfectados` o el encabezado `X-Registros-Eliminados`).

7. Instrucciones de instalación

Requisitos

- JDK 26 e IntelliJ IDEA (o Maven Wrapper incluido en `backend/`)
- Node.js 20 o superior (incluye npm)
- MySQL 8 o superior, en ejecución
- Git

Pasos

1. Clonar el repositorio

```
git clone https://github.com/Progra4-2026/Sistema_Web_Gimnasio-Grupo_1.git
cd Sistema_Web_Gimnasio-Grupo_1
```

2. Crear la base de datos (crea `gimnasio_db`, las tablas y los datos de prueba)

```
mysql -u root -p < database/GIMNASIO.sql
```

Si la base ya existía de un entregable anterior, se vuelve a ejecutar el script: agrega las tablas `administradores` y `parametros_sistema` y la columna `instructores.password_hash`.

En PowerShell: `Get-Content database\GIMNASIO.sql | mysql -u root -p`

3. Configurar las credenciales de MySQL (solo si no son root/root): copiar `backend/.env.example` como `backend/.env` y completar `DB_URL`, `DB_USER` y `DB_PASSWORD`.

4. Levantar el backend: abrir `backend/` en IntelliJ con el JDK 26 y ejecutar `GimnasioBackendApplication`, o desde la terminal:

```
cd backend
./mvnw spring-boot:run
```

Verificación: `http://localhost:8080/api/health` responde `{"status":"ok"}`.

5. Levantar el frontend (en otra terminal)

```
cd frontend
npm install
npm run dev
```

Abrir `http://localhost:5173`.

Usuarios de prueba

Al arrancar, `CargaInicialDatosRunner` asigna a los clientes e instructores de prueba que no tengan contraseña su propia cédula o código (cifrada con BCrypt) y crea un administrador de demostración si no existe ninguno. Nunca reemplaza una contraseña existente.

Rol	Usuario (cédula / código)	Contraseña
Cliente	`200` (o cualquier cédula sembrada)	su propia cédula
Instructor	`100` (o cualquier código sembrado)	su propio código
Administrador	`1`	`admin123`

Rol	Sección del frontend después de iniciar sesión
Cliente	Servicios → Rutinas (`/servicios/rutinas`): solo sus rutinas
Instructor	Panel instructor (`/instructor/rutinas` y `/instructor/maquinas`)
Administrador	Mantenimiento (`/mantenimiento`): cursos, clientes, instructores, administradores y parámetros

Problemas frecuentes

Síntoma	Solución
`Unknown database 'gimnasio_db'` o `Unknown column 'password_hash'`	Ejecutar el paso 2 (script GIMNASIO.sql).
`package org.springframework.security… does not exist`	Recargar el proyecto Maven (Maven → Reload) para descargar `spring-boot-starter-security`.
`records are not supported in -source 8`	IntelliJ compila con nivel de Java 8: en Settings → Build Tools → Maven → Runner, activar “Delegate IDE build/run actions to Maven” y elegir el JDK 26.
El frontend muestra “No se pudo conectar con el servidor”	El backend no está levantado en el puerto 8080.
Error de CORS en la consola del navegador	El frontend se abrió desde un origen que no está en `app.cors.origenes-permitidos`; agregarlo en application.properties.